SIMATS ENGINEERING
CO3-ASSESSMENT TOOL1- CASE STUDY

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA1709**

COURSE OUTCOME COVERED

CO3: *Analyze knowledge representation and reasoning using propositional logic, FOL, inference, resolution, chaining methods, knowledge representation to perform logical reasoning for drawing valid conclusions and decision making. (BL4)*

Assessment Tool Weightage: 40%

QUESTIONS: 4 **Total Marks:40**

Code of Conduct:

I V.Madhulika(192312620) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

V.Madhulika
Signature of the student

Annexure A

CASE STUDY

Case Study 1 — Smart Medical Diagnosis System

A hospital is designing a rule-based Logical Agent for preliminary patient diagnosis. The system's knowledge base contains:

I. If a patient has Fever AND Cough → Possible Flu

Ii. If a patient has Fever AND Rash → Possible Measles

Iii. If a patient has Cough AND Breathlessness → Possible Pneumonia

Iv. Facts about Patient A: Fever = True, Cough = True, Breathlessness = True

(a) Represent the above rules and facts precisely using Propositional Logic symbols. Define each symbol used.

(b) Apply Forward Chaining on the knowledge base to derive all possible diagnoses for Patient A. Show each inference step with the rule fired and fact derived.

(c) The goal is to confirm 'Patient A has Pneumonia.' Apply Backward Chaining and draw the goal reduction tree to verify this goal.

Case Study 2 — Autonomous Traffic Management Agent

A smart city traffic controller uses a FOL-based knowledge base to manage emergency vehicle routing:

I. ∀x: Vehicle(x) ∧ EmergencyType(x) → ClearPath(x) [Rule 1]

II. ∀x: ClearPath(x) → GreenSignal(x) ∧ ¬RedSignal(x) [Rule 2]

III. ∀x ∀y: Vehicle(x) ∧ Behind(x,y) ∧ GreenSignal(y) → Proceed(x) [Rule 3]

IV. Facts: Vehicle(Ambulance), EmergencyType(Ambulance), Vehicle(CarA), Behind(CarA, Ambulance)

(a) Apply Unification to match Rule 1 with the given facts. Show the complete substitution set θ (MGU) at each step.

(b) Using the Resolution method, prove that 'Proceed (CarA)' is true. Convert relevant clauses to CNF and show each resolution step.

(c) Analyze whether the current FOL knowledge base is sufficient to handle two simultaneous emergency vehicles. Identify what additional rules or facts are needed and justify logically.

Case Study 3 — Agricultural Expert Reasoning System

An agricultural advisory logical agent operates with the following propositional knowledge base (KB):

1) P1: SoilDry → IrrigationNeeded
2) P2: IrrigationNeeded ∧ CropWheat → ApplyDripMethod
3) P3: ¬ApplyDripMethod ∧ CropWheat → CropAtRisk
4) Facts: SoilDry = True, CropWheat = True

(a) Convert all sentences P1, P2, P3 and the facts into Conjunctive Normal Form (CNF). Show each conversion step (eliminate implication → move ¬ inward → distribute).

(b) Using Resolution Refutation, prove the theorem 'ApplyDripMethod'. Negate the goal, combine with the KB in CNF, and derive the empty clause □. Draw the resolution proof tree.

(c) If the fact 'SoilDry' is removed from the KB, analyze step-by-step how logical conclusions change. Does 'CropAtRisk' still follow? Apply appropriate reasoning patterns to justify.

Case Study 4 — Wumpus World Knowledge Agent

An AI agent navigating the Wumpus World perceives: Stench at [1,2], Breeze at [1,1], Glitter at [2,2]. The agent uses a propositional KB with these rules:

1) Stench [1,2] → Wumpus is adjacent to cell [1,2]
2) Breeze [1,1] → Pit is adjacent to cell [1,1]
3) Glitter [x,y] → Gold is at [x,y]
4) ¬Wumpus [x,y] ∧ ¬Pit[x,y] → Safe[x,y]

(a) Construct the agent's belief state from its percepts. Apply Modus Ponens systematically to derive new knowledge facts. List all derived conclusions.

(b) Using Forward Chaining from the agent's percepts, identify the possible locations of the Wumpus and Pit. Show each reasoning step precisely.

(c) Evaluate whether the path [1,1] → [1,2] → [2,2] is safe for the agent to collect the gold. Use the knowledge derived in (a) and (b) to justify your conclusion with logical inference.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

Case Study 1 — Smart Medical Diagnosis System

A hospital is designing a rule-based Logical Agent for preliminary patient diagnosis. KB: (i) Fever ∧ Cough → Possible Flu; (ii) Fever ∧ Rash → Possible Measles; (iii) Cough ∧ Breathlessness → Possible Pneumonia. Facts about Patient A: Fever = True, Cough = True, Breathlessness = True.

(a) Propositional Logic Representation

Symbols defined:

- F = Patient has Fever
- C = Patient has Cough
- R = Patient has Rash
- B = Patient has Breathlessness
- Flu = Patient possibly has Flu
- Measles = Patient possibly has Measles
- Pneumonia = Patient possibly has Pneumonia

Rules:

R1: F ∧ C → Flu
R2: F ∧ R → Measles
R3: C ∧ B → Pneumonia

Facts (Patient A): F = True, C = True, B = True (R not asserted → treated as False/unknown)

(b) Forward Chaining — Deriving All Diagnoses for Patient A

Known facts (initial): {F, C, B}

Step 1: Check R1: F ∧ C → both F and C are TRUE → RULE FIRES
Derived fact: Flu | Known facts now: {F, C, B, Flu}

Step 2: Check R2: F ∧ R → F is TRUE but R is unknown/False → RULE DOES NOT FIRE

Step 3: Check R3: C ∧ B → both C and B are TRUE → RULE FIRES
Derived fact: Pneumonia | Known facts now: {F, C, B, Flu, Pneumonia}

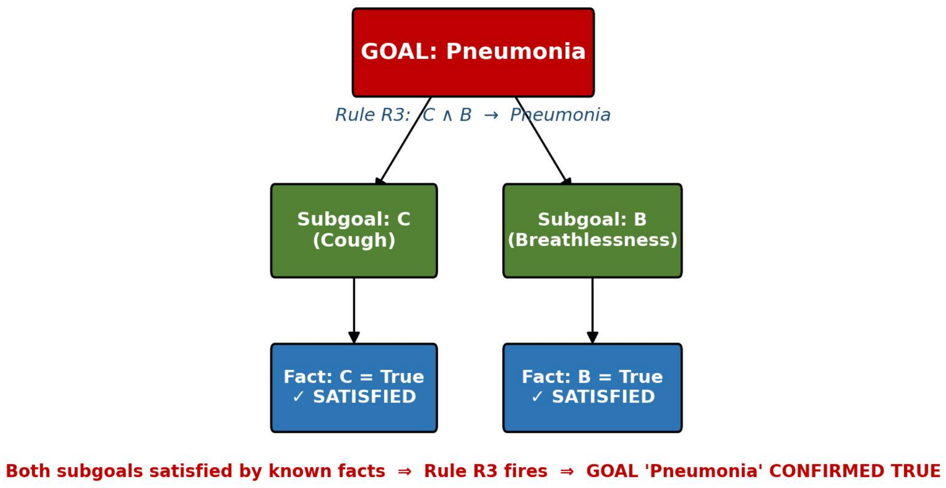
Step 4: Re-scan all rules — no further rule has all premises satisfied. STOP.

Final diagnoses derived for Patient A: Possible Flu and Possible Pneumonia (Measles is not indicated, since Rash was never asserted).

(c) Backward Chaining — Confirming Goal 'Patient A has Pneumonia'

Goal: Pneumonia. Find a rule concluding Pneumonia → R3: C ∧ B → Pneumonia. Reduce goal to sub-goals {C, B} and check each against the known facts.

Case Study 1(c) — Backward Chaining Goal Reduction Tree



Case Study 2 — Autonomous Traffic Management Agent

KB — Rule 1: ∀x: Vehicle(x) ∧ EmergencyType(x) → ClearPath(x). Rule 2: ∀x: ClearPath(x) → GreenSignal(x) ∧ ¬RedSignal(x). Rule 3: ∀x∀y: Vehicle(x) ∧ Behind(x,y) ∧ GreenSignal(y) → Proceed(x). Facts: Vehicle(Ambulance), EmergencyType(Ambulance), Vehicle(CarA), Behind(CarA, Ambulance).

(a) Unification of Rule 1 with the Facts

Rule 1 (general form): Vehicle(x) ∧ EmergencyType(x) → ClearPath(x)

Step 1: UNIFY Vehicle(x) with fact Vehicle(Ambulance)
Substitution: θ1 = {x / Ambulance}

Step 2: Apply θ1 to the remaining literal EmergencyType(x)
→ EmergencyType(Ambulance)
This matches the fact EmergencyType(Ambulance) exactly — consistent, no conflict.

Most General Unifier: θ = {x / Ambulance}

Result: Rule 1 instantiates to Vehicle(Ambulance) ∧ EmergencyType(Ambulance) → ClearPath(Ambulance)
→ ClearPath(Ambulance) can be derived.

(b) Resolution Proof that 'Proceed(CarA)' is True

Step 1 — Convert relevant sentences to CNF:

Rule 1: Vehicle(x) ∧ EmergencyType(x) → ClearPath(x)
CNF → ¬Vehicle(x) ∨ ¬EmergencyType(x) ∨ ClearPath(x) ... C1

Rule 2: ClearPath(x) → GreenSignal(x) ∧ ¬RedSignal(x) (split conjunctive consequent)
CNF → ¬ClearPath(x) ∨ GreenSignal(x) ... C2
CNF → ¬ClearPath(x) ∨ ¬RedSignal(x) ... C2'

Rule 3: Vehicle(x) ∧ Behind(x,y) ∧ GreenSignal(y) → Proceed(x)
CNF → ¬Vehicle(x) ∨ ¬Behind(x,y) ∨ ¬GreenSignal(y) ∨ Proceed(x) ... C3

Facts (unit clauses):

F1: Vehicle(Ambulance)   F2: EmergencyType(Ambulance)
F3: Vehicle(CarA)   F4: Behind(CarA, Ambulance)

Negated Goal: ¬Proceed(CarA) ... NG

Step 2 — Resolution steps (refutation):

1. Resolve C1 {¬Vehicle(x) ∨ ¬EmergencyType(x) ∨ ClearPath(x)} with F1 {Vehicle(Ambulance)}
θ={x/Ambulance} → {¬EmergencyType(Ambulance) ∨ ClearPath(Ambulance)} ... R1

2. Resolve R1 with F2 {EmergencyType(Ambulance)}
→ {ClearPath(Ambulance)} ... R2

3. Resolve C2 {¬ClearPath(x) ∨ GreenSignal(x)} with R2 {ClearPath(Ambulance)}
θ={x/Ambulance} → {GreenSignal(Ambulance)} ... R3

4. Resolve C3 {¬Vehicle(x) ∨ ¬Behind(x,y) ∨ ¬GreenSignal(y) ∨ Proceed(x)} with F3 {Vehicle(CarA)}
θ={x/CarA} → {¬Behind(CarA,y) ∨ ¬GreenSignal(y) ∨ Proceed(CarA)} ... R4

5. Resolve R4 with F4 {Behind(CarA,Ambulance)}
θ={y/Ambulance} → {¬GreenSignal(Ambulance) ∨ Proceed(CarA)} ... R5

6. Resolve R5 with R3 {GreenSignal(Ambulance)}
→ {Proceed(CarA)} ... R6

7. Resolve R6 {Proceed(CarA)} with NG {¬Proceed(CarA)}
→ EMPTY CLAUSE □

Empty clause derived ⇒ the negated goal leads to a contradiction ⇒ Proceed(CarA) is PROVED TRUE.

(c) Is the KB Sufficient for Two Simultaneous Emergency Vehicles?

No — the current KB is not sufficient. Rule 1 grants ClearPath(x) to every vehicle that independently satisfies Vehicle(x) ∧ EmergencyType(x), with no check for interaction between two emergency vehicles. If two ambulances approach a shared/conflicting intersection at the same time, Rule 1 and Rule 2 would independently derive ClearPath and GreenSignal for both — which is logically consistent within the KB but physically unsafe/contradictory in the real world, since conflicting approaches cannot both safely receive a green signal at the same junction.

Additional rules/facts needed:

- A predicate ConflictingRoute(x, y) — true when the paths of vehicles x and y intersect at a junction, used to detect when two emergency vehicles compete for the same right-of-way.
- A priority predicate Priority(x, y) — encoding which emergency vehicle should be given precedence (e.g., based on arrival time, severity/type of emergency, or distance to the junction).
- A modified Rule 1 / new Rule 1b: ∀x∀y: Vehicle(x) ∧ Vehicle(y) ∧ EmergencyType(x) ∧ EmergencyType(y) ∧ ConflictingRoute(x,y) ∧ Priority(x,y) → ClearPath(x) ∧ ¬ClearPath(y), ensuring only the higher-priority vehicle receives ClearPath (and therefore GreenSignal) when two emergency vehicles conflict.

Justification: Without these additions, the KB can derive two mutually incompatible conclusions (GreenSignal for two conflicting directions) without any logical contradiction being flagged, because nothing in the current axioms represents the real-world mutual-exclusion constraint between conflicting green signals.

Case Study 3 — Agricultural Expert Reasoning System

KB — P1: SoilDry → IrrigationNeeded. P2: IrrigationNeeded ∧ CropWheat → ApplyDripMethod. P3: ¬ApplyDripMethod ∧ CropWheat → CropAtRisk. Facts: SoilDry = True, CropWheat = True.

(a) Conversion of P1, P2, P3 and Facts to CNF

P1: SoilDry → IrrigationNeeded
Step 1 (eliminate →): ¬SoilDry ∨ IrrigationNeeded
Step 2 (¬ already innermost, no distribution needed)
CNF: ¬SoilDry ∨ IrrigationNeeded ... C1

P2: (IrrigationNeeded ∧ CropWheat) → ApplyDripMethod
Step 1 (eliminate →): ¬(IrrigationNeeded ∧ CropWheat) ∨ ApplyDripMethod
Step 2 (De Morgan, ¬ inward): ¬IrrigationNeeded ∨ ¬CropWheat ∨ ApplyDripMethod
Step 3 (already a disjunction of literals — no distribution needed)
CNF: ¬IrrigationNeeded ∨ ¬CropWheat ∨ ApplyDripMethod ... C2

P3: (¬ApplyDripMethod ∧ CropWheat) → CropAtRisk
Step 1 (eliminate →): ¬(¬ApplyDripMethod ∧ CropWheat) ∨ CropAtRisk
Step 2 (De Morgan, ¬ inward): (ApplyDripMethod ∨ ¬CropWheat) ∨ CropAtRisk
Step 3 (flatten disjunction):
CNF: ApplyDripMethod ∨ ¬CropWheat ∨ CropAtRisk ... C3

Facts (already unit clauses):
F1: SoilDry F2: CropWheat

(b) Resolution Refutation — Prove 'ApplyDripMethod'

Negate the goal: ¬ApplyDripMethod (NG). Combine with the CNF KB {C1, C2, C3, F1, F2} and resolve until the empty clause is derived.

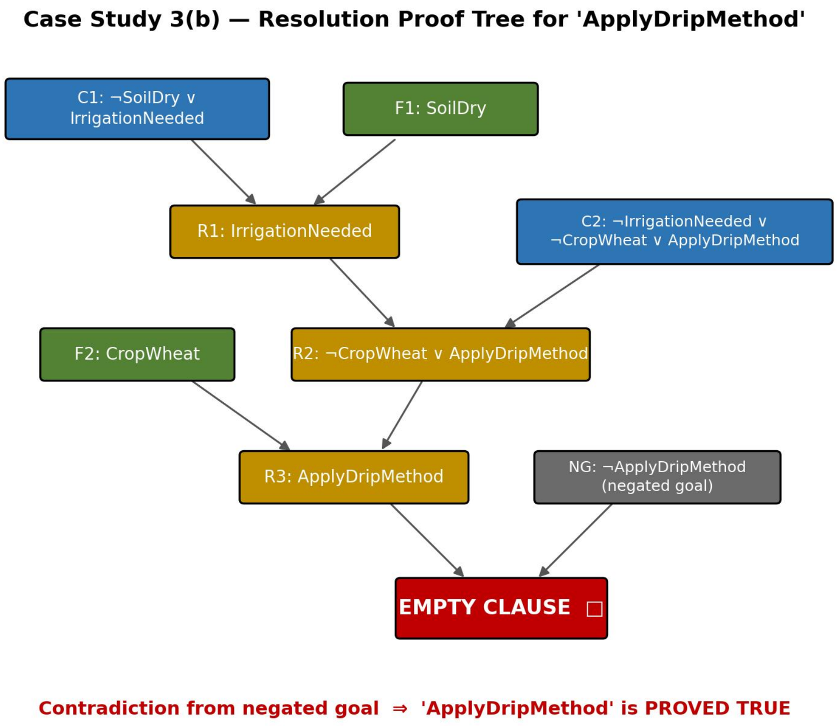
1. Resolve C1 {¬SoilDry ∨ IrrigationNeeded} with F1 {SoilDry}
→ {IrrigationNeeded} ... R1

2. Resolve C2 {¬IrrigationNeeded ∨ ¬CropWheat ∨ ApplyDripMethod} with R1 {IrrigationNeeded}
→ {¬CropWheat ∨ ApplyDripMethod} ... R2

3. Resolve R2 with F2 {CropWheat}
→ {ApplyDripMethod} ... R3

4. Resolve R3 {ApplyDripMethod} with NG {¬ApplyDripMethod}
→ EMPTY CLAUSE □

Empty clause derived ⇒ contradiction from the negated goal ⇒ ApplyDripMethod is PROVED TRUE.



(c) Effect of Removing the Fact 'SoilDry'

Without the unit clause SoilDry, C1 {¬SoilDry ∨ IrrigationNeeded} can no longer be resolved to derive {IrrigationNeeded}, since SoilDry is no longer known true. Consequently:

- IrrigationNeeded can no longer be derived, so C2 can no longer be resolved down to ApplyDripMethod either — ApplyDripMethod becomes logically undetermined (neither provably true nor provably false) under classical open-world reasoning.
- Under strict classical logic (no closed-world assumption), since ¬ApplyDripMethod cannot be proven either, the premise of P3 (¬ApplyDripMethod ∧ CropWheat) cannot be established as true — so CropAtRisk CANNOT be logically derived from the KB at all. The system reaches no conclusion about crop risk.
- Under the Closed World Assumption (CWA) — common in practical rule-based/production expert systems — any fact that cannot be proven true (such as ApplyDripMethod here) is treated as false by default. In that case ¬ApplyDripMethod is assumed true, and since CropWheat is still a fact, P3 fires and CropAtRisk IS derived — but now via a different reasoning path (default 'irrigation was not confirmed applied' rather than an explicit proof that it was withheld).

Conclusion: Whether 'CropAtRisk' still follows depends on the reasoning convention adopted — it does NOT follow under strict open-world resolution (the KB becomes simply insufficient to decide), but it DOES follow under the Closed World Assumption typically used in expert systems, illustrating why non-monotonic/default reasoning patterns are important in practical rule-based agents.

Case Study 4 — Wumpus World Knowledge Agent

Percepts: Stench[1,2], Breeze[1,1], Glitter[2,2]. KB Rules: (1) Stench[1,2] → Wumpus adjacent to [1,2]; (2) Breeze[1,1] → Pit adjacent to [1,1]; (3) Glitter[x,y] → Gold at [x,y]; (4) ¬Wumpus[x,y] ∧ ¬Pit[x,y] → Safe[x,y]. The agent starts at [1,1] (a cell it occupies safely). On a standard grid, [1,1] (corner) has neighbors {[1,2],[2,1]}; [1,2] (edge) has neighbors {[1,1],[1,3],[2,2]}.

(a) Belief State via Modus Ponens

Premise: Stench[1,2]   Rule 1: Stench[1,2] → WumpusAdjacent[1,2]

Modus Ponens ⇒ WumpusAdjacent[1,2] is TRUE

⇒ Wumpus ∈ { [1,1], [1,3], [2,2] }   (the neighbours of [1,2])

Premise: Breeze[1,1]   Rule 2: Breeze[1,1] → PitAdjacent[1,1]

Modus Ponens ⇒ PitAdjacent[1,1] is TRUE

⇒ Pit ∈ { [1,2], [2,1] }   (the neighbours of [1,1])

Premise: Glitter[2,2]   Rule 3: Glitter[x,y] → Gold[x,y]

Modus Ponens ⇒ Gold[2,2] is TRUE

Derived conclusions: (i) Wumpus is at [1,1], [1,3] or [2,2]; (ii) Pit is at [1,2] or [2,1]; (iii) Gold is confirmed at [2,2].

(b) Forward Chaining — Locating the Wumpus and Pit

Step 1: Fact Stench[1,2] fires Rule 1 → candidate set Wumpus ∈ {[1,1], [1,3], [2,2]}

Step 2: Fact Breeze[1,1] fires Rule 2 → candidate set Pit ∈ {[1,2], [2,1]}

Step 3: The agent currently occupies [1,1] and has survived → [1,1] contains neither Wumpus nor Pit (else the agent would already be lost).

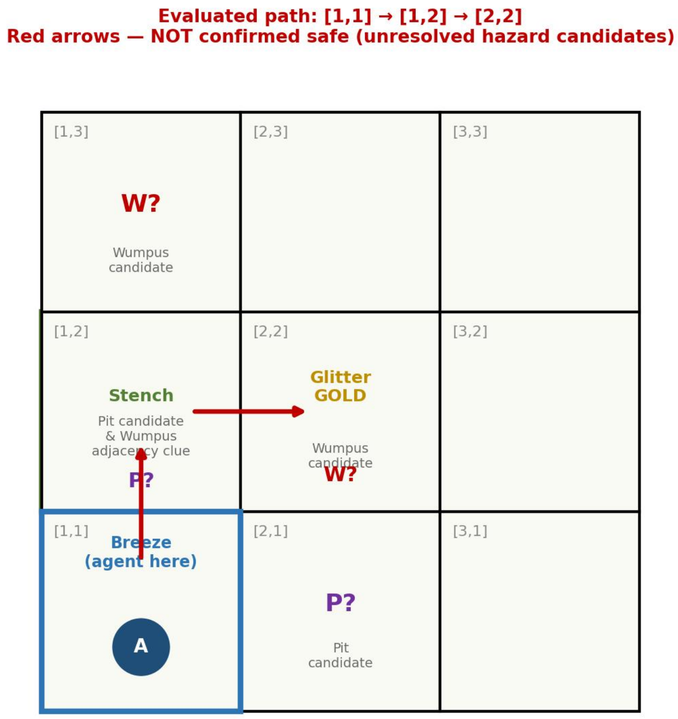
⇒ Eliminate [1,1] from the Wumpus candidate set

⇒ Refined: Wumpus ∈ {[1,3], [2,2]}

Step 4: No Breeze percept has been reported at any cell other than [1,1], and no Stench percept has been reported at any cell other than [1,2]. With only these two percepts available, [1,2] remains implicated as BOTH a Pit candidate (Step 2) — the most uncertain cell in the current belief state.

Result: Wumpus is most likely at [1,3] or [2,2]; Pit is most likely at [1,2] or [2,1]. No cell can yet be reduced to full certainty with only these three percepts.

Case Study 4 — Wumpus World Belief State & Evaluated Path



(c) Is the Path [1,1] → [1,2] → [2,2] Safe?

Apply Rule 4 (¬Wumpus[x,y] ∧ ¬Pit[x,y] → Safe[x,y]) to each cell on the proposed path, using the candidate sets derived in (a) and (b):

- [1,1]: Confirmed Safe — the agent occupies it without harm, so ¬Wumpus[1,1] ∧ ¬Pit[1,1] both hold. Rule 4 fires → Safe[1,1].
- [1,2]: Cannot be confirmed Safe. It is a member of the Pit candidate set {[1,2],[2,1]} derived in (a)/(b), so ¬Pit[1,2] cannot be established. Since one of Rule 4's premises (¬Pit[1,2]) is not proven, Safe[1,2] CANNOT be derived.
- [2,2]: Cannot be confirmed Safe. It is a member of the (refined) Wumpus candidate set {[1,3],[2,2]}, so ¬Wumpus[2,2] cannot be established. Rule 4 again fails to fire, so Safe[2,2] CANNOT be derived — despite [2,2] also being the confirmed Gold location.

Conclusion: The path [1,1] → [1,2] → [2,2] CANNOT be logically certified as safe with the current knowledge base and percepts. Both [1,2] (possible Pit) and [2,2] (possible Wumpus, though holding the Gold) remain unresolved hazard candidates. A rational agent should not commit to this path without first gathering additional percepts (e.g., visiting [2,1] or [1,3] to test for Breeze/Stench and shrink the candidate sets) or applying a probabilistic risk-weighting between the reward of the Gold and the danger of an unconfirmed hazard, since pure propositional inference leaves the path's safety logically undecided.